Wide but Shallow

Technical-Interview Problems Are Four Ideas in Many Costumes

and company-specific preparation is mostly a myth

Conor Garvey Gelvin
`gelvin@outlook.com`

Data snapshot: June 2026

Abstract

Interview-prep advice rests on two claims: that different companies test different skills, and that you must memorize dozens of separate problem "patterns." We test both against how often each problem appears among seven major technology firms' most-asked lists (769 distinct problems), and both mostly fail. The dozens of "patterns" you are told to learn are really **four underlying ideas**: a single left-to-right pass (a *fold*), divide-and-solve recursion (dynamic programming), spreading information across a graph until it settles, and binary search. To put a number on this without hand-waving, we take a *pre-registered* random sample of 100 problems and, for each, prove with the Lean proof assistant that a standard accepted solution really is one of the four ideas. **71 of the 100 carry such a proof** (69 machine-checked here, 2 citing existing formal proofs); the other 29 are a genuine "tail" that fits no single idea. Every classification was checked against the problem's published editorial. So about **three-quarters of interview problems reduce to four ideas**, each backed by a machine-checked proof, and the single pass is the most common—many tricks taught as unrelated (prefix sums, the XOR trick, a hash "seen set," reversing a list, the sliding window) are the *same* pass, differing only in what running value they carry. Separately, the seven firms ask almost the same mix: on a 0-to-1 scale of how differently they ask (bias-corrected Cramér's V, where 0 is identical), they score just 0.091, above the 0.065 that sampling noise alone would produce but far below "moderate"—six of the seven are statistically indistinguishable, and only Uber stands out (it leans on graph problems). We describe which problems are *commonly asked*, not how hard they are or whether company-specific drilling pays off. In short: interview problems are *wide on the surface but shallow underneath*, and essentially one shared syllabus covers nearly every company. All derived data and proofs are released; the raw scrape is withheld per the platform's terms.

1 Two myths

Two beliefs dominate interview preparation. The first is *company specificity*: that Google, Meta, Amazon and their peers test different skills, so you should prepare differently for each. The second is *pattern proliferation*: that the field is a long list of loosely related tricks—sliding window, monotonic stack, union–find, prefix sums, and so on—each learned separately, as popular pattern catalogues present them [1]. Both are widely repeated and, as far as we can tell, never tested.

We test them, and then ask what underlying *structure* explains the answer. The short version: the "dozens of patterns" you are told to memorize are a handful of ideas in different disguises, and the seven companies ask nearly the same things. To keep our labels honest we do not hand-sort the problems and stop there—we use a proof assistant to check, problem by problem, that the idea we assign really does solve the problem (and, for most of them, that the solution is fully correct).

Underlying idea	in scheme	formal test (Lean)	textbook families it subsumes
streaming fold	27	`IsFold`	two-pointers, sliding-window, monotonic-stack, hashing, prefix-sum, bit-XOR, string-scan
recursive decomposition (DP)	25	catamorphism	dynamic programming, backtracking, tree, trie
graph relaxation	11	least fixpoint	BFS/DFS, Dijkstra, Bellman–Ford, topo-sort, union–find
bisection	8	monotone threshold	binary search (monotone predicate)
one of the four	**71**		
not a scheme	—		design, simulation, heaps, number theory, geometry, string matching, ...

Table 1: The roughly twenty surface families collapse onto four recursion schemes. On a pre-registered sample of 100 problems, 71 are proven to be one of the four (69 machine-checked here, 2 cited [10, 12]; both graph relaxation); 29 form the genuine tail. "Formal test" names the Lean predicate each certificate proves membership in. Full method and numbers: Sections 3–4.

2 Data and method

For seven firms (Amazon, Apple, Bloomberg, Google, Meta, Microsoft, Uber) we scraped a public interview-prep platform (LeetCode) for how often each problem appears among that firm's *most-asked* problems (the platform exposes a top-200 list per firm and recency window, so rarely-asked problems are truncated; we note this where it matters). Across firms this covers 769 distinct problems, broken down by recency. The platform tags each one with a *surface family* ("sliding window," "monotonic stack," "dynamic programming," and so on), which we consolidate into about twenty broad families.

We map each problem to one of four underlying *ideas* (formally, *recursion schemes*) by the *shape* of its standard solution, or to a leftover "tail" for problems that fit no single idea (Table 1). We ask two separate questions and keep them apart. *Coverage*—what fraction of problems the four ideas explain—is settled per problem by a machine-checked proof, not by a label: on a pre-registered random sample we prove, for each problem, that a standard accepted solution really is the assigned idea (Section 4); the coverage figure is the fraction that go through. (A written-in-advance family-to-idea rule, Appendix A, supplies the initial guess of which idea to try—it is a starting point for the proofs, not the basis of the count.) Separately, the *company* comparison—do firms ask different mixes?—weights each idea by how often it is asked (Section 3); the comparison is over a seven-firm by four-idea table of asked-problem counts (n = 1,400); it groups by the four ideas, with the finer twenty-family view alongside as a robustness check, and all difference measures are bias-corrected [3] and reported with uncertainty.

3 Result: four ideas, not dozens

Wide surface. The surface taxonomy really is spread out: no one or two families dominate (it takes six of the ∼20 families to cover even half of asked problems; counting each distinct problem once instead, the spread is similarly flat—Gini 0.30, where 0 is a perfectly even split and 1 is total concentration). By the usual measure, interview preparation *looks* like a long, flat list. This is the breadth the prep industry sells.

Shallow root. But that breadth is mostly repackaging. Sorting the same problems by the *shape* of their standard solution, about three-quarters reduce to just four ideas—**71 of a random sample of 100, each backed by a machine-checked proof** (Section 4; 95% Wilson interval $[61\%, 79\%]$)—with the single left-to-right pass the most common. Tricks memorized as unrelated—prefix sums, the XOR trick, a hash "seen set," reversing a list, matching parentheses with a stack, the sliding window—are the *same* pass; they differ only in what running value they carry. The variety is in the bookkeeping, not in the computation. The remaining quarter—bespoke data structures, simulation, geometry, ad-hoc number theory—is a genuine tail we do not force into a scheme.

One shared syllabus. The same data dissolve the company myth. We score how differently the firms ask on a standard 0-to-1 scale (bias-corrected Cramér's V [2, 3], where 0 means identical and the usual cutoffs call 0.1 "small" and 0.3 "moderate"). Across the four ideas the firms score just $V = 0.091$ (95% bootstrap interval $[0.06, 0.12]$)—small, with the whole interval well below "moderate," so the difference is genuinely bounded, not merely undetected. We report the four-idea grouping because its score stands clear of the small difference that random chance alone would produce in a sample this size (0.091 against a chance level of 0.065); at the finer twenty-family level the score falls *below* its own chance level—each grouping has one, set by its number of categories—(0.064 against 0.116)—indistinguishable from identical—so the finer view, if anything, makes the firms look more alike, not less. Comparing firms two at a time, the only real differences all involve **Uber**, which asks more graph problems and fewer single-pass ones (its largest gap, versus Google, is still only "small"; Figure 1); **the other six firms are statistically indistinguishable**. One caveat: we compare the *mix of problem types*, not how hard the problems are or whether drilling for a specific company helps—our data cannot measure those. Within that scope, there is essentially one interview syllabus, and preparing differently per company is, for most firms, wasted effort.

4 Machine-checking the classification

A classification is only as good as its labels, and the usual check—a second human rater—only tells you whether two people read a label the same way. We do something stronger: we settle each label with a *proof assistant*, a tool that mechanically verifies mathematical claims. Working in Lean 4 [4] with its library Mathlib [5], we write each of the four ideas as a precise test on programs (for example, "this function is a single left-to-right pass"). Then, for each sampled problem, we model a standard accepted solution and prove two things: that it has the *shape* of its idea, and that it satisfies a property of the *specific* problem—its actual grid, array, or condition. The second part is what gives the proof teeth: a statement about an abstract relation would certify the *category*, not the problem. So every certificate is held to a mechanical standard: a released Lean program (`lake exe gate`) recomputes each verdict directly from the compiled proofs, and no human judgement decides whether a finished certificate counts. It checks three things. Both theorems must be statements about the modelled solution itself. The certificate must evaluate that solution on a concrete input

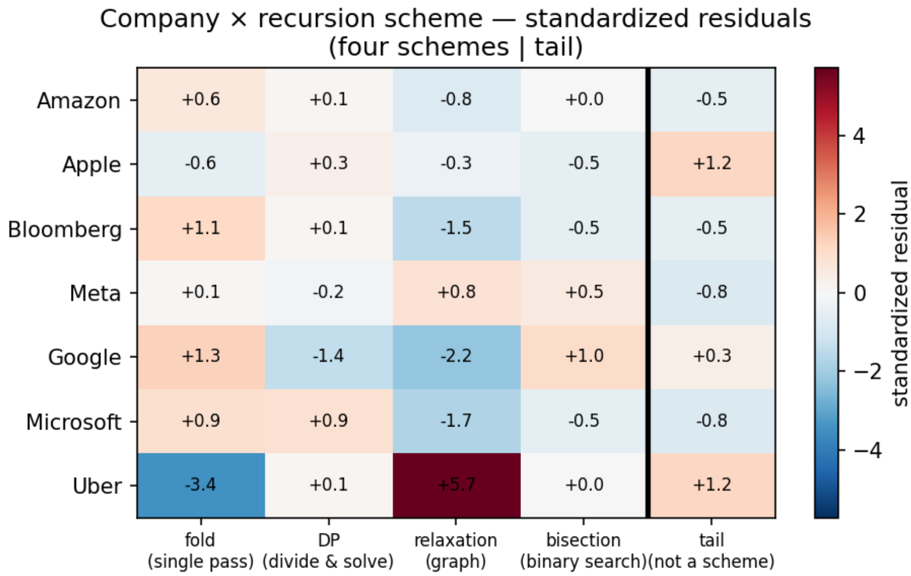

Figure 1: How much each firm over- or under-asks each idea, relative to the shared average (standardized residuals; red = more than expected, blue = less). Only Uber stands out—it asks far more graph problems (+5.7) and far fewer single-pass ones (−3.4)—while the other six rows are muted (all within about ±2). This is the picture behind the small overall difference ($V = 0.091$): one shared syllabus, with Uber the lone exception.

— usually the problem's published example — as a kernel-checked theorem, something no abstract placeholder can produce. And every proof must rest on Lean's standard axioms alone, which rules out unfinished proofs and shortcuts that trust the compiler rather than the proof checker.

Two further theorems pin down the most common scheme, the streaming fold. First, being a single pass is a property of the function, not of how its code happens to be written: a function is a streaming fold exactly when its answer on a longer input is determined by its answer so far plus the one new element (`isFold_iff_update`) — one sweep, carrying the answer as it goes. Second, the scheme can be *failed*: a sweep whose carried value is just the running count provably cannot count distinct elements (`not_isFold_countDistinct`)—that task needs a richer carried value, the "seen set" of Table 1—and even computing a list's length, which is a sweep, provably cannot be done with any fixed amount of memory (`length_not_finiteStateFold`). "Everything is a fold" is therefore false as a theorem, not merely denied. We never demand a proof that the algorithm is *optimal*—the platform's acceptance already vouches for the submissions themselves, and our question is only *which idea solves the problem*—though most certificates prove exact correctness anyway (49 of 69, below). A problem counts when both proofs pass with no gaps (Lean flags any unfinished proof; there are none) and use only Lean's standard axioms.

A pre-registered sample. Which problems land in the sample matters, because some are far easier to formalize than others. An earlier draft drew from a seed we had, in effect, chosen well—we had proved that one sample to near-completion, which measures how hard we worked, not how often the four ideas hold (scored afterward by how many of its problems our then-existing proofs covered, it ranked first out of 2,000 random draws). We discarded it, fixed a new seed *before drawing* (recorded in the released `PREREGISTRATION.md`), and report whatever it yields. On that sample, **71 of 100 problems** are proven to be one of the four ideas; the other 29 are the genuine tail. Of the 71, **69 are machine-checked here**; the remaining two—finding a minimum spanning tree (LC 1489) and finding the bridges of a graph (LC 1192)—rest on a classic algorithm whose correctness is itself a substantial formalization, so rather than redo it we cite an existing machine-checked proof [10, 12] that the algorithm is the graph computation we classify it as.

Checked against the editorials. The proofs are about solutions we *model*; to guard against modelling a convenient strawman, we cross-checked every candidate against the problem's published editorial and confirmed the standard accepted solution really is the idea we assigned. Three needed a note: two were a different one of the four than our first guess (we relabelled them—both still in scope), and one (Add Digits) we moved to the tail, because its *optimal* accepted solution is a constant-time arithmetic formula outside the four—a single-pass solution also passes, but we count it conservatively as tail. That move is why the count is 71, not 72.

What this does and does not show. We prove that a solution of the right shape solves the problem; we do not prove it is byte-for-byte the platform's accepted code (our model captures the solution's structure, not its every line). So a certificate says "this problem really is solved by idea S," not "the accepted submission is literally this program." How much each proof shows: **49 of the 69 prove the exact answer or a complete characterization** — the search finds a thing if and only if it is really there, an operation's entire effect is captured in one equation, or the reported value is achieved by a real path or plan and beaten by none. The remaining 20 prove an exact law of the modelled solution; in each of those the model *is* the specification — the very recurrence, tally, or condition the accepted solution computes — so there is nothing further to prove about it. The submissions themselves are vouched for by the platform's acceptance, which we never re-litigate. Two of those 20 — cycle detection and longest palindromic subsequence — *are* counted among the 69, because their scheme membership and problem-specific laws are machine-checked here; only the textbook correctness of the underlying classical algorithms is taken from work we cite rather than redo (Floyd's cycle-finding, including locating the cycle's start [11]; the optimal longest-common-subsequence recursion [9]). They are distinct from LC 1489 and LC 1192 above, the two problems we do *not* count as machine-checked here. This bar earns its keep: demanding the full balanced-string equivalence for bracket matching exposed a lenient model that silently accepted a stray closing bracket — the strict checker now carries the full equivalence, machine-checked. And we do not re-prove the deep theory behind the ideas—that shortest paths and dynamic programming are formally well-founded is already established [6, 7, 8, 9], which we cite rather than redo.

5 Related work

Two literatures bracket this paper. On the *empirical* side, interview preparation is organized by pattern catalogues [1] that enumerate surface families; we are not aware of prior work that tests whether those families are generatively distinct, or whether firms differ in the mix they ask. On the *structural* side, the observation that wide families of programs are instances of a few recursion schemes is the Bird–Meertens tradition of program calculation [13] and its formal-methods descendants: associative aggregation over a sliding window is a verified fold [14], dynamic programming is verified memoized recursion [9], and shortest paths are a least fixpoint over a semiring [6, 7, 8]; verified-algorithms textbooks now develop these schemes end to end [15, 16]. The four-scheme decomposition itself is therefore not new—it is the classical Bird–Meertens repertoire. Our contribution is to connect that structural lens to the *empirical* distribution of interview problems (the company-comparison test), and to machine-check the scheme assignment per problem rather than assert it.

6 Threats to validity

One platform. The frequencies come from a single prep platform and may reflect its own collection process as much as what employers truly ask; we describe *commonly-asked* problems, not any firm's private bar. **Top-200 cap.** Each firm's frequencies are its *most-asked* top-200 list per recency window, not the full distribution, so the rare tail is truncated; the company comparison is over the heads of the distributions, where the data are densest. **Borrowed labels.** The family-to-idea rule is ours, but the surface-family tags it starts from are the platform's—single-source and unaudited—so the "wide surface" count inherits whatever splits that taxonomy chose. **We model the solution, not the submission.** The proofs are about a solution we write down to capture the standard accepted approach, not the platform's exact code; we checked every candidate against the published editorials to guard against strawmen, but "this captures the accepted approach" remains a modelling judgement, not something the proof itself certifies. **Which idea is a choice.** The proofs certify that an idea solves the problem (and, per `not_isFold_countDistinct`, that this is a claim a problem can provably fail), not that it is the only sensible description. **A single-sample estimate.** 71% comes from one pre-registered sample, so the 95% interval $[61\%, 79\%]$, not the point value, is the honest summary; it errs low if anything, since the four ideas absorb some borderline "greedy"/"simulation" problems we left in the tail. **Window dependence.** The company difference is not constant across recency windows: the bias-corrected V ranges from below its chance level (computed per window) in the short windows up to 0.125 in the six-month window (still "small"), so "one shared syllabus" is a statement about the pooled data, not every slice. **Two separate questions.** Coverage counts each distinct problem once; the company comparison weights by how often a problem is asked. Both matter to a candidate, and we keep them apart. **Shallow is not easy.** "Same idea" does not mean "same difficulty"—two single-pass problems can differ hugely in how hard the running value is to find. We claim shallow *structure*, not that the problems are easy.

7 Conclusion

Interview problems are wide on the surface but shallow underneath. On a pre-registered random sample, 71 of 100 problems (69 with a machine-checked proof here, 2 citing existing formal proofs) are one of four ideas (the single left-to-right pass is the most common); the rest are a genuine tail. And the seven firms ask essentially one shared syllabus. For a candidate, the takeaway is blunt: learn the four ideas deeply rather than dozens of patterns shallowly, and prepare for "technical interviews," not for any one company.

Reproducibility and AI-use disclosure

All derived statistics, the pre-registration of the sampling seed (`PREREGISTRATION.md`, committed before the sample was drawn or scored), the per-problem proof list (from which the 71/100 re-derives), the editorial cross-check, and the complete Lean development are released at `https://github.com/corsur/swarm-tips/tree/main/research/wide-but-shallow`. The gate that decides which proofs count is a released Lean program (`lake exe gate`) that recomputes every verdict from the compiled proofs themselves (Section 4), so the count is reproducible rather than a matter of our judgement. The raw per-company frequency tables are withheld under the platform's terms, so the headline numbers reproduce from the released derived data even though the scrape itself is not redistributed. The proofs build with `lake build` with no gaps; the "standard axioms" claim is checkable, not merely asserted, by running `#print axioms` on the main theorems. This paper was written by one human author with substantial AI assistance for drafting, statistical code, and the formalization; all claims, numbers, and proofs were checked by the author, and the machine-checked results are independently verifiable by re-running the proofs. We say so plainly: the paper's central claims stand or fall on the released data and the proofs, not on how it was written.

References

[1] S. Prashad. LeetCode Patterns. `https://seanprashad.com/leetcode-patterns/`, 2020.

[2] H. Cramér. *Mathematical Methods of Statistics*. Princeton University Press, 1946.

[3] W. Bergsma. A bias-correction for Cramér's *V* and Tschuprow's *T*. *Journal of the Korean Statistical Society*, 42(3):323–328, 2013.

[4] L. de Moura and S. Ullrich. The Lean 4 theorem prover and programming language. In *Automated Deduction (CADE-28)*, 2021.

[5] The mathlib Community. The Lean mathematical library. In *Proc. 9th ACM SIGPLAN Int. Conf. on Certified Programs and Proofs (CPP)*, 367–381, 2020.

[6] B. Nordhoff and P. Lammich. Dijkstra's shortest path algorithm. *Archive of Formal Proofs*, 2012.

[7] A. Armstrong, G. Struth, and T. Weber. Kleene algebra. *Archive of Formal Proofs*, 2013.

[8] D. J. Lehmann. Algebraic structures for transitive closure. *Theoretical Computer Science*, 4(1):59–76, 1977.

[9] S. Wimmer, S. Hu, and T. Nipkow. Monadification, memoization and dynamic programming. *Archive of Formal Proofs*, 2018.

[10] M. P. L. Haslbeck, P. Lammich, and J. Biendarra. Kruskal's algorithm for minimum spanning forest. *Archive of Formal Proofs*, 2019.

[11] P. Gammie. The Tortoise and the Hare algorithm. *Archive of Formal Proofs*, 2015.

[12] P. Lammich and R. Neumann. A framework for verifying depth-first search algorithms. *Archive of Formal Proofs*, 2016; see also *Proc. CPP*, 137–146, 2015.

[13] R. Bird and O. de Moor. *Algebra of Programming*. Prentice Hall, 1997.

[14] L. Heimes, D. Traytel, and J. Schneider. Formalization of an algorithm for greedily computing associative aggregations on sliding windows. *Archive of Formal Proofs*, 2020.

[15] T. Nipkow, J. Blanchette, M. Eberl, A. Gómez-Londoño, P. Lammich, C. Sternagel, S. Wimmer, and B. Zhan. *Functional Algorithms, Verified!* 2021. `https://functional-algorithms-verified.org`.

[16] T. Nipkow, M. Eberl, and M. P. L. Haslbeck. Verified textbook algorithms: A biased survey. In *Automated Technology for Verification and Analysis (ATVA)*, 2020.

A Initial family-to-idea guess

To decide which idea to *try proving* for each problem, we start from a written-in-advance map from surface family to idea, by the shape of the canonical accepted solution: a single left-to-right pass carrying one accumulator → *streaming fold*; a recurrence that decomposes a structure into sub-results → *recursive decomposition (DP)*; iterative improvement toward a fixed point over a graph → *graph relaxation*; a monotone predicate located by halving → *bisection*; anything else (bespoke data structures, simulation, geometry, ad-hoc number theory, string matching) → *tail*. This is only the starting guess; the per-problem Lean proof (Section 4), checked against the editorial, is what settles each classification, and a few problems were relabelled to a different one of the four when the editorial disagreed with the initial guess.